Amazon AMI — Console Dashboard Wise Important Interview Questions & Answers

EC2
│
├── AMIs
│ ├── Owned by me
│ ├── Public images
│ ├── AWS Marketplace AMIs
│ └── Shared with me
│
├── Instances
│ └── Instance
│ ├── AMI / Image
│ ├── Instance type
│ ├── Storage
│ ├── Security
│ └── Network
│
├── Snapshots
│
└── Images
├── AMIs
└── AMI-related settings

1. What is an AMI?
Answer:
AMI stands for Amazon Machine Image. It is a template used to launch EC2 instances.
An AMI can contain the operating system, application software, configuration, and required storage information.
Instead of configuring every new EC2 instance manually, we can launch instances from the same AMI.
Example:
Suppose we configure one EC2 server with Linux, Java, and our application.
We create an AMI from that configured server.
Later, we can use the AMI to launch multiple EC2 instances with the same setup.
Diagram:
Configured EC2
↓
Create AMI
↓
AMI
↓
Launch EC2
├── EC2-1
├── EC2-2
└── EC2-3
Simple ga gurthupettuko:
AMI = EC2 launch template/image

---

2. Why do we use an AMI?
Answer:
We use AMIs to create EC2 instances with a predefined configuration.
They help us avoid repeating operating system and application setup manually.
AMIs are especially useful when we need multiple identical servers or quick recovery.
Example:
Suppose we have a web server configured with Nginx and our application.
Instead of installing everything again on every EC2 instance, we create an AMI once.
We can then launch new servers from that AMI whenever required.
Diagram:
Configured Server
↓
AMI
↓
New EC2 Instances
Simple ga gurthupettuko:
AMI = Ready-made EC2 setup

---

3. What is an AMI ID?
Answer:
Every AMI has a unique identifier called an AMI ID.
The AMI ID is used to identify the exact image we want to use when launching an EC2 instance.
It is usually displayed in a format such as ami-xxxxxxxx.

Example:
Suppose we have an AMI containing our application configuration.
When launching EC2, we select that AMI ID so AWS knows which image should be used.
This ensures we launch the intended image.
Diagram:
AMI
↓
AMI ID
↓
ami-xxxxxxxx
↓
Launch EC2
Simple ga gurthupettuko:
AMI ID = Unique ID of AMI

---

4. What information does an AMI contain?
Answer:
An AMI contains information required to launch an EC2 instance.
This includes the operating system image and block device mapping, including information about the root volume and additional volumes.
It can also represent software and configuration that existed when the image was created.
Example:
Suppose our EC2 server has Linux, application files, and a root EBS volume.
When we create an AMI, the required image and storage configuration are captured.
New instances can then be launched using that configuration.
Diagram:
AMI
├— OS
├— Software / Configuration
└— Block Device Mapping
  ├— Root Volume
  └— Additional Volumes
Simple ga gurthupettuko:
AMI = OS + Configuration + Storage mapping

---

5. What is the difference between AMI and EC2 Instance?
Answer:
An AMI is a template used to launch EC2 instances, while an EC2 instance is the running virtual server.
The AMI provides the initial configuration for the instance.
After launching, the instance can be modified independently.
Example:
Suppose we have one AMI containing Ubuntu and our application.
We use it to launch three EC2 instances.
The AMI is the template, while those three running servers are instances.
Diagram:
AMI
↓
Launch
↓
EC2 Instance
↓
Running Server
Simple ga gurthupettuko:
AMI = Template
EC2 = Running server

AMI Console — Owned by Me
6. What is "Owned by me" in the AMI console?
Answer:
"Owned by me" shows the AMIs owned by our AWS account.
These can be AMIs that we created ourselves or AMIs that were copied into our account.
We can use these AMIs to launch EC2 instances based on their configuration.
Example:
Suppose we configure an application server and create an AMI from it.
That AMI appears under the AMIs owned by our account.
We can use it whenever we need another identical application server.
Diagram:
My AWS Account
↓
Owned AMIs
├— App-AMI

├── Web-AMI
└── Test-AMI

Simple ga gurthupettuko:
Owned by me = Mana account AMIs

---

7. What are Public AMIs?
Answer:
Public AMIs are AMIs that are available for other AWS users to use.
AWS and other publishers can provide public AMIs for common operating systems and software.
Before using one, we should verify the publisher and image details.
Example:
Suppose we want to launch a Linux server quickly.
Instead of creating our own image, we can select a suitable public AMI.
We then launch an EC2 instance using that image.
Diagram:
Public AMI
↓
Select Image
↓
Launch EC2
Simple ga gurthupettuko:
Public AMI = Publicly available image

---

8. What are AWS Marketplace AMIs?
Answer:
AWS Marketplace provides AMIs containing software products from AWS and third-party vendors.
Some products may require additional software licensing or subscription charges.
They allow us to launch preconfigured software instead of installing it manually.
Example:
Suppose a company needs a preconfigured security or database product.
Instead of installing the software manually, we can select its Marketplace AMI.
The EC2 instance is launched with the vendor's software.
Diagram:
AWS Marketplace
↓
Software AMI
↓
EC2
↓
Preconfigured Application
Simple ga gurthupettuko:
Marketplace AMI = Ready software from marketplace

---

9. What does “Shared with me” mean for AMIs?
Answer:
“Shared with me” refers to AMIs that another AWS account has shared with our account.
The owner controls the AMI, while the receiving account can use it according to the sharing configuration.
This is useful when organizations need to share standardized server images.
Example:
Suppose the central IT account creates a company-approved AMI.
It can share that AMI with application accounts.
Those accounts can then launch EC2 instances using the approved image.
Diagram:
Account A
↓
Company AMI
↓
Share AMI
↓
Account B
↓
Launch EC2
Simple ga gurthupettuko:
Shared AMI = Another account share chesina image

Creating an AMI
10. How do you create an AMI from an EC2 instance?
Answer:
We can create an AMI from an existing EC2 instance through the EC2 console or AWS APIs.
AWS captures the required image and storage information needed to launch instances from the AMI.
The resulting AMI can then be used to launch new EC2 instances.

Example:
Suppose our EC2 server has Java, Nginx, and our application already configured.
We create an AMI from that instance.
Later, we can launch new EC2 servers with the same basic setup.
Diagram:
EC2
↓
Create Image
↓
AMI
↓
Launch New EC2
Simple ga gurthupettuko:
EC2 → Create Image → AMI

---

11. What happens when we create an AMI from an EC2 instance?
Answer:
AWS creates an image definition and captures the required EBS-backed storage information for the instance.
The AMI stores the configuration needed to launch another EC2 instance.
Depending on the instance storage configuration, the associated snapshot resources are also involved.
Example:
Suppose our EC2 instance has a root EBS volume containing the operating system.
When we create an AMI, AWS creates the required image and snapshot-based resources.
The AMI can then be used to launch another instance.
Diagram:
EC2
↓
AMI Creation
↓
AMI
↓
EBS Snapshot(s)
↓
New EC2
Simple ga gurthupettuko:
AMI creation = Image + required storage snapshots

---

12. What is an EBS-backed AMI?
Answer:
An EBS-backed AMI uses EBS snapshots as the backing storage for the instance's root volume and potentially additional volumes.
Most modern EC2 instances commonly use EBS-backed AMIs.
When a new instance is launched, EBS volumes are created from the AMI's storage mapping.
Example:
Suppose our AMI uses an EBS snapshot containing the Linux root volume.
When we launch an EC2 instance, AWS creates the root EBS volume from that snapshot.
The instance then boots using that volume.
Diagram:
AMI
↓
EBS Snapshot
↓
EBS Volume
↓
EC2
Simple ga gurthupettuko:
EBS-backed AMI = AMI → Snapshot → EBS Volume

AMI Storage / Snapshots
13. Does an AMI store the actual EBS volume?
Answer:
An AMI does not simply store a live EBS volume as-is.
For EBS-backed AMIs, it contains the image information and block device mapping that references the snapshots used to create the volumes.
When launching an instance, AWS creates volumes from the relevant snapshots.
Example:
Suppose the AMI references a snapshot containing our operating system.
When we launch a new EC2 instance, AWS uses that snapshot to create a new EBS root volume.
The new instance gets its own volume.
Diagram:
AMI
↓

Snapshot
↓
New EBS Volume
↓
EC2
Simple ga gurthupettuko:
AMI = Snapshot references + launch configuration

---

14. What is block device mapping in an AMI?
Answer:
Block device mapping defines the storage devices that should be attached when an EC2 instance is launched from an AMI.
It can specify the root volume and additional EBS volumes.
It helps determine how storage is configured for the new instance.
Example:
Suppose our application needs one root volume and one additional data volume.
The AMI can contain the required block device mapping.
When we launch an instance, AWS creates and attaches the specified volumes.
Diagram:
AMI
↓
Block Device Mapping
├── /dev/root → Root Volume
└── /dev/data → Data Volume
↓
EC2
Simple ga gurthupettuko:
Block Device Mapping = EC2 storage layout

AMI Permissions
15. Can we share an AMI with another AWS account?
Answer:
Yes, an AMI can be shared with specific AWS accounts when supported by the AMI configuration.
The receiving account can then use the shared AMI to launch instances.
Sharing an AMI does not transfer ownership of the AMI to the receiving account.
Example:
Suppose our central AWS account creates a standard application AMI.
We can share that AMI with a development account.
The development team can then launch EC2 instances from the shared image.
Diagram:
Account A
↓
AMI
↓ Share
Account B
↓
EC2
Simple ga gurthupettuko:
Share AMI = Use cheyyachu, ownership transfer avvadu

---

16. Can we make an AMI public?
Answer:
An AMI can be made public when the configuration and ownership requirements allow it.
A public AMI can be visible and usable by other AWS accounts.
Before making an AMI public, we must ensure it does not contain sensitive information or credentials.
Example:
Suppose a company creates a generic software image intended for public use.
It can publish the AMI publicly after removing secrets and private information.
Other AWS users can then discover and use it.
Diagram:
Private AMI
↓
Make Public
↓
Public AMI
↓
Other AWS Accounts
Simple ga gurthupettuko:
Public AMI = Other users can use it

---

17. Can we copy an AMI to another Region?

Answer:
Yes, supported AMIs can be copied from one AWS Region to another.
The copied AMI becomes an image available in the destination Region.
This is useful for disaster recovery and launching applications closer to users.
Example:
Suppose our application AMI is available in Mumbai.
We can copy it to Singapore so that we can launch the same application there.
This helps with multi-Region deployments and recovery planning.
Diagram:
Mumbai
↓
AMI
↓ Copy
Singapore
↓
Copied AMI
↓
EC2
Simple ga gurthupettuko:
AMI Copy = Region-to-Region image copy

---

18. What is the difference between AMI copy and AMI sharing?
Answer:
AMI sharing allows another account to use an AMI while the original owner retains the AMI.
AMI copying creates a new AMI in the destination account or Region, depending on the operation.
The new copy can then be managed independently by its owner.
Example:
Suppose Account A owns an AMI and shares it with Account B.
Account B can use the shared AMI.
If Account B needs its own independent image, it can copy the AMI when supported.
Diagram:
Sharing:
Account A → AMI → Account B

Copying:
Account A → AMI → New AMI
Simple ga gurthupettuko:
Share = Same AMI use
Copy = New AMI

AMI Management
19. What is deregistering an AMI?
Answer:
Deregistering an AMI removes the AMI registration so that it can no longer be used to launch new instances through that AMI.
For EBS-backed AMIs, the underlying snapshots are separate resources and are not automatically removed just because the AMI is deregistered.
We should check for dependent snapshots before deleting unused image resources.
Example:
Suppose we have an old application AMI that is no longer required.
We can deregister the AMI to remove it from our available images.
We should separately review its snapshots if they are no longer needed.
Diagram:
AMI
↓
Deregister
↓
AMI unavailable for new launches

Snapshots
↓
Remain separate
Simple ga gurthupettuko:
Deregister AMI ≠ Automatically delete all snapshots

---

20. Why should we remove old AMIs?
Answer:
Old AMIs may no longer be required and can make image management difficult.
Their associated snapshots can also continue to consume storage and incur charges if they are retained.
We should keep required recovery images and remove obsolete ones according to the organization's retention policy.
Example:
Suppose we have 20 old versions of an application AMI.

Only the latest few versions are required for deployment and rollback.
We can remove obsolete AMIs and their unnecessary snapshots after confirming they are no longer needed.
Diagram:
Old AMIs
├── AMI-v1 ❌
├── AMI-v2 ❌
├── AMI-v3 ✅
└── AMI-v4 ✅
↓
Cleanup
Simple ga gurthupettuko:
Old AMI cleanup = Better image + snapshot management

AMI Security
21. What should we check before sharing an AMI?
Answer:
Before sharing an AMI, we should check whether the image contains passwords, access keys, private certificates, or other sensitive information.
We should also verify the software and configuration included in the image.
Only approved accounts or users should receive access to the image.
Example:
Suppose our AMI contains an application server.
Before sharing it, we remove hardcoded credentials and verify that no private keys are stored in the image.
Then we share the image only with the required AWS accounts.
Diagram:
AMI
↓
Security Check
├── Credentials
├── Private Keys
├── Secrets
└── Configuration
↓
Share Safely
Simple ga gurthupettuko:
AMI sharing = First remove secrets

AMI + Auto Scaling
22. How is an AMI used with an Auto Scaling Group?
Answer:
An Auto Scaling Group can use an AMI through its launch configuration or launch template.
When the ASG needs new EC2 instances, it launches them using the configured AMI.
This allows newly launched instances to have a consistent operating system and application setup.
Example:
Suppose our application runs on EC2 and traffic increases.
The Auto Scaling Group launches additional instances using our application AMI.
Each new server starts with the same basic application configuration.
Diagram:
AMI
↓
Launch Template
↓
Auto Scaling Group
↓
EC2-1
EC2-2
EC2-3
Simple ga gurthupettuko:
AMI + ASG = Same setup for new servers

AMI + Disaster Recovery
23. How can AMIs help with disaster recovery?
Answer:
AMIs can preserve a known server configuration that can be used to launch replacement EC2 instances.
We can maintain appropriate AMIs in required Regions as part of a disaster recovery strategy.
This can reduce the time required to recreate application servers.
Example:
Suppose our production EC2 server fails or a Region-level recovery plan is required.
We can use a prepared AMI in the recovery Region to launch a replacement server.
The application configuration can therefore be restored faster.

Diagram:
Production
↓
AMI
↓
Copy to DR Region
↓
DR AMI
↓
Launch EC2
Simple ga gurthupettuko:
AMI = Fast EC2 recovery

AMI + Launch Template
24. What is the relationship between AMI and Launch Template?
Answer:
An AMI defines the image from which the EC2 instance starts, while a Launch Template defines broader launch configuration.
A Launch Template can specify the AMI ID along with instance type, networking, security groups, storage, and other settings.
Together they provide a repeatable way to launch EC2 instances.
Example:
Suppose our application needs a specific AMI, instance type, security group, and subnet configuration.
We can put these settings into a Launch Template.
An Auto Scaling Group can then use that template to launch consistent EC2 instances.
Diagram:
AMI
+
Launch Template
↓
Auto Scaling Group
↓
EC2
Simple ga gurthupettuko:
AMI = Image
Launch Template = Full launch configuration

AMI Important Console Flow
25. How do you launch an EC2 instance from an AMI?
Answer:
First, we select the required AMI from the EC2 AMIs section.
Then we choose the instance type, networking, storage, security groups, and other required settings.
Finally, we launch the instance, and AWS creates the EC2 server using the selected AMI.
Example:
Suppose we have an application AMI containing Ubuntu and our application.
We select that AMI, choose an appropriate instance type and security group, and launch it.
The new EC2 instance starts using the AMI configuration.
Diagram:
EC2 Console
↓
AMIs
↓
Select AMI
↓
Choose Instance Settings
↓
Launch
↓
EC2
Simple ga gurthupettuko:
AMIs → Select → Configure → Launch EC2